

Benchmarking Two Techniques for Tor Classification

Flow Level and Circuit Level Classification

Khalid Shahbar A. Nur Zincir-Heywood

Faculty of Computer Science
Dalhousie University
Halifax, Canada
{Shahbar, Zincir}@cs.dal.ca

Abstract— Recently, many internet users, who seek anonymity, use Tor, which is one of the most popular anonymity software solutions. Tor provides this anonymity by hiding the identity of the user from the destination that the user aims to reach. It also hides the user activities into encrypted cells. In this work, we investigate up to what level we can define what the user in Tor is doing. To this end, we extended on the previous work to classify the user activities using information extracted from Tor circuits and cells. Moreover, we developed a classification system to identify user activities based on traffic flow features. Our results show that flow based classification can reach up to the accuracy of the cell level classification as well as being more flexible.

Keywords—Tor traffic classification; Traffic Flow; Tor Circuits

I. INTRODUCTION

Tor provides its users with anonymity by hiding their browsing activities and locations. This level of anonymity achieved by relaying the users activities through three Tor nodes. When the user connects to the first relay, the first relay knows the IP address of this user. The user activities stay encrypted while moving from the first relay to the second one, and from the second one to the third one. The third relay sends the user activities on behalf of the user without any information regarding the identity of the user.

In this work, we aim to analyze the amount of information that can be extracted from the encrypted Tor traffic without decrypting the traffic. To explore this, we propose to employ machine learning (ML) algorithms. To this end, we employ two different approaches for the classification of user activities. The first approach is the Flow level classification. It depends on analyzing the TCP communication between the user and the relay to predict the type of user activities in the encrypted traffic. The second approach is the Circuit level classification. The encrypted circuits have characteristics that can be extracted and calculated to classify the type of traffic in the circuits. The main difference between the Flow level classification and the Circuit level classification is the level of access to Tor network. The Circuit level classification requires having access to the relay node, itself [1]. That means only the one who runs the relay can gather the required statistics about the circuit. On the other hand, the Flow level classification requires access to anywhere between the user and the relay.

This means the collection of the encrypted TCP communication between the user and the relay can be in the user machine, the relay machine, or the ISP.

Our work provides a benchmarking study for the classification of Tor user activities. Furthermore, we extend the prior work [1] for classifying Tor circuits to achieve high classification accuracy, and aim to classify Tor flows with high accuracy even without having access to the relay.

The reminder of this paper is structured as follows. Section II presents the related work. Tor background is reviewed in section III. Features used in both the Flow and the Circuit level classification are explained in section IV. Experiments performed and the results obtained are presented in sections V and VI, respectively. Finally, conclusions are drawn and the future work is discussed in section VII.

II. RELATED WORK

In [1], Alsabah et al. used ML algorithms to classify the application used by Tor's users. The applications are Browsing, Streaming and BitTorrent. Because Tor traffic is encrypted, they used the circuit level and the cell level information to do the classification. The circuit level information includes the circuit lifetime and the amount of data transferred by the circuit. The cell level information includes the inter arrival time of the cells and their statistics. The classification includes online and offline classification. The online classification uses the cell level information to classify the circuit while it is in use. The offline classification uses both the cell and the circuit level information to classify the circuit. In terms of accuracy, the best result they achieved in the offline classification is 91%, whereas the best results achieved in the online classification is 97.8%.

In [2], a simulated Tor network is used to differentiate between encrypted traffic and Tor traffic. The simulated network contains three directory servers and fifteen relays. Salenium browser (a Firefox add-on) is used to gather a sample of thirty website browsing traces. To do the comparison between the two types of encrypted traffic, HTTPS traffic is gathered alone for a period of two weeks then followed by HTTP traffic over the simulated Tor network for two weeks. Lastly, an HTTPS over the simulated network is gathered also for two weeks. In total, around 25 GB of data collected for the

three different tests. Each sample is 1 MB in size, so to build the whole captured data set, mergcap is used. Different ML algorithms (Random forest, J4.8 and Adaboost) were employed via Weka [21]. Their results showed that more than 90% of the HTTP and HTTPS traffic over Tor network could be detected with a false positive rate of 3.7%.

In [3], a support vector machine (SVM) classifier is used with the websites' fingerprints to show that Tor and JAP are not providing total anonymity. Volume, time and direction of the traffic are the features in SVM. In the closed world (set of known websites), the training data set consists of browsing different websites using a computer running Debian distribution in Tor network or JAP and logging the features using TCPdump. Chickenfoot Firefox plugin is used to automate the browsing and retrieving activities. The data set contains 775 websites with 20 instances for each of them. On the other hand, in the open world (unknown websites) part of their experiments, the training data set includes 4000 URL chosen from the 1 million most popular websites provided by Alexa [15]. Another 1000 URLs are added to the test data. The features used in both data sets are: (i) Packets size (52 bytes): the size of the packet that contains the ACK between the sender and the receiver; (ii) Size marker: a special text label that shows the change in the direction of the packet. It is used to sum up all packets in the same direction; (iii) HTML marker: to distinguish between each HTML request and to check the size of the requested site; (iv) Rounded transmitted bytes: the number of all transmitted packets in both directions (rounded in increments of 10,000); (v) Number of markers: grouping the packet size for all packets of a flow in one direction; (vi) Occurring packet size: the number of each site's packets in both directions; (vii) Percentage of Incoming Packets; and (viii) Number of packets. Using these features improves the results of detecting closed world websites from 3% to 55% in Tor network and from 20% to 80% in JAP compared to other related works. In the open world data set, the detection is 73% with 0.05% false positive rate.

In [4], authors use traffic analysis to discover the identity of the user using the Tor network. The analysis depends on the size of the packet transmitted on the network from the web server through the router to the user. Tor has a fixed cell size but the packet size can vary. So they used padding with one bit to mark the packets so that they could be traced back at the receiver side. The authors report that ten packets are enough to get a reasonable detection with low false positive. The length of the padding will force the Tor router to use the known number of cells. The Tor cell size is 512 bytes, which means if the data size is more than 512 bytes then it needs to be fragmented to fit into the cell size. This enables the authors to mark the client who receives these cells as the client has access to the server. During the path from the server to the client, packets may face congestion, retransmission or any normal traffic behavior that affects the receiver to detect the encoded bit. To solve such problems, delay is used between buffered packets before sending them. By setting the delay interval to 400 ms, the authors achieved a detection rate of 90% with 4% false positive rate.

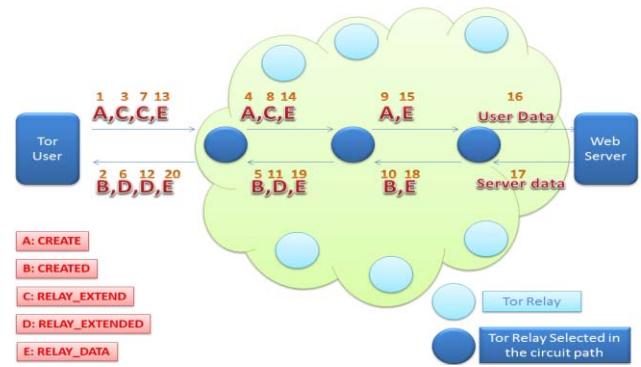


Fig. 1. The Sequence for Establishing a Tor Circuit.

III. TOR BACKGROUND

Tor is a publicly available software that is built to provide users with anonymization when using the Internet. Tor Network depends on the volunteers who run their machines as relay nodes to forward other users' traffic through Tor. The user starts by establishing a virtual circuit. This circuit consists of three relays. A directory server is used to get the addresses of available routers and their keys to encrypt the data. User data is encrypted with three layers of encryption. First, data are encrypted with the last relay (exit router) key then the middle relay and finally, by the first relay (entry router). This way, the entry router only knows the source of the data. The middle router(s) only knows where to get the encrypted data and where to send them, but nothing about the source of the data or the destination of them. Finally, the exit router knows the destination of the data, but knows nothing about the source. This ensures that the whole path is not known by any of the three nodes (routers) on the Tor network.

The user and the relay use encrypted TLS connections to communicate. Circuits and cells are the components for Tor communication. The circuit is a virtual path from the user machine to the last relay (the exit relay). Inside the circuit the data are packed into cells. Each cell contains the identifier of the circuit it belongs. The circuit ID is unique in the connection between the user and the relay. Through the path, the circuit ID is not the same.

Figure 1 shows the sequence for establishing the circuit. When the user wants to send data through Tor, he/she starts by establishing a circuit. The circuit establishment starts from the user to the first relay in a CREATE cell. When the circuit is established, the relay sends back the CREATED cell to the user. The user then extends the circuit to the second relay by using the RELAY_EXTEND cell. The first relay sends the CREATE cell to the second relay. The second relay responds back with the CREATED cell when the circuit is established between the first and the second relay. The first relay replies back to the user with the RELAY_EXTENDED cell to inform the user that the circuit path is now up to the second relay.

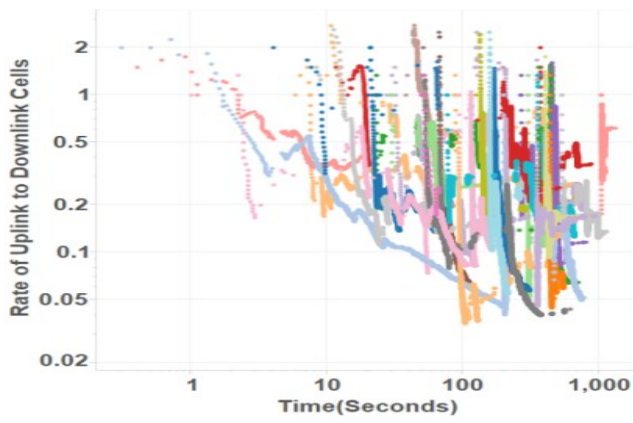


Fig. 2. Browsing Circuits Data Rate - Different colors show the rate for different circuits.

Then in the same way, the user extends the circuit to the exit relay. After the circuit is established between the user and the exit relay, the user is ready to send the data into RELAY_DATA cells.

The data travel in both directions between the user and the exit relay. This is only an overview of how the circuit is established in Tor to start sending the data. There are other forms and versions of command and relay cells explained in detail in [6].

Even though the cells in the circuit are encrypted, they have different behaviors and characteristics according to the type of the traffic they carry. Cells timing, the number of downlink cells, the number of uplink cells, and the number of cells in the duration of the circuit vary based on the type of the traffic in the cells. There is a directly proportional relation between the type of traffic and the number of cells that flow in the circuit. The higher the amount of data the higher the number of cells. The uplink cells that flow from the user to the relay depend on how the application acts to send data requests.

Figure 2 shows how the rate of uplink cells to the downlink cells changes with time in web browsing circuits. In most of the circuits, the rate reaches to its maximum value. This rate only stays for a short period then it starts to decay quickly. The rate then starts to fluctuate up and down until it reaches a very low rate. This observation shows conformity with normal web browsing requests where the user enters a URL and waits for the web site to download then enters another URL and so on. The amount of data in the requests sent is lower than the amount of data downloaded from the web sites themselves. This causes the rate to decay with time.

Figure 3 shows how the rate behaviors differ for the video streaming circuits. The rate is higher than the web browsing. The decay period is longer and smoother. The rate then stays in a certain range for the rest of the circuit duration time. The BitTorrent circuits show yet another behavior in the rate as shown in Figure 4. In this case, the rate reaches to 100 and decays slowly. The circuits duration is longer than both the web browsing and the video streaming. BitTorrent uplink cells

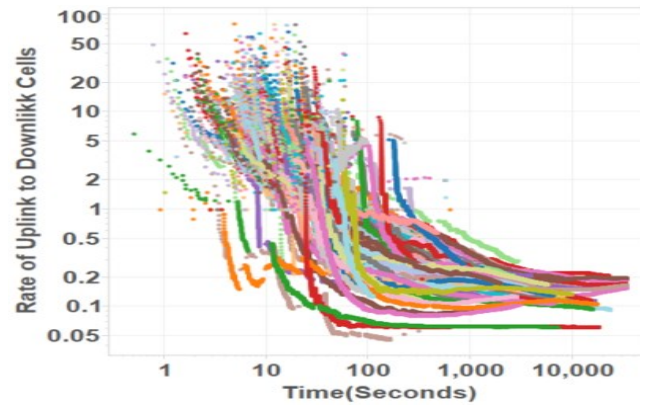


Fig. 3. Streaming Circuits Data Rate - Different colors show the rate for different circuits.

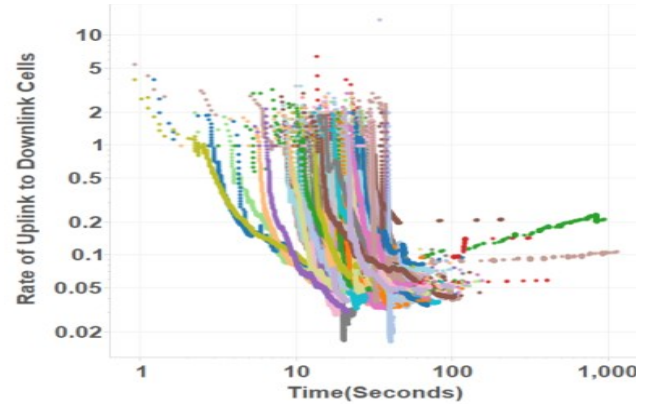


Fig. 4. BitTorrent Circuit Data Rate - Different colors show the rate for different circuits.

stay high in numbers compared to the web browsing or the streaming. However, because the number of the downlink cells are high in BitTorrent, the rate decays with time.

IV. CLASSIFICATION FOR TOR

A. Circuit Level Classification

This approach depends on using statistics extracted from the Circuits and their Cells to classify the user activities in the Tor traffic without decrypting the traffic. It is similar to the work done in [1] but we extended it with the following attributes:

1) *Cells per circuit life time*: The amount of data delivered by a circuit is a good indication about the type of traffic carried by the circuit. But without considering the time the circuit is alive, this indication might be misleading. For example, two circuits might deliver the same amount of data in two different durations. As shown in Figure 5, the number of streaming circuits and BitTorrent circuits are both high compared to the number of browsing circuits. The circuit life time is what distinguishes between them.

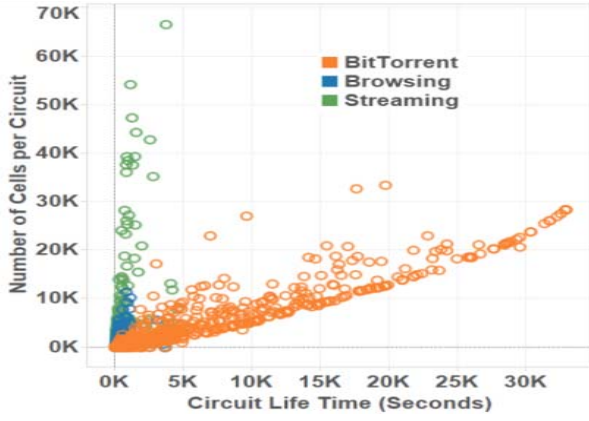


Fig. 5. The Number of Cells During Circuit Life Time.

2) *Uplink Cells*: Relays' cells are transferred in both directions; uplink and downlink. BitTorrent seems to transmit higher amount of uplink cells than the browsing or streaming applications. In other words, number of cells required to send a request might be a good indicator of the type of traffic running on the Tor network.

3) *The Rate of the Downlink Cells to the Uplink Cells*: Even though the uplink data is a good indication about the type of traffic the circuit has, the rate of the uplink to the downlink is also important. If the circuit life time is long enough, it could transmit considerable amount of uplink cells even if it is used for browsing. However, most of the time, the rate will vary based on the type of traffic no matter how long the life of the circuit is. Figure 6 shows the rate of the downlink cells to the uplink cells with the circuit life time.

4) *Exponentially Weighted Moving Average (EWMA)*: Intuitively, we think that EWMA is a good indicator because it can differentiate between two circuits when both have the same life time. We used the following formula to calculate the EWMA of the cells [7]:

$$\bar{x}_k = \alpha \bar{x}_{k-1} + (1 - \alpha) x_k$$

Where $\bar{x}_k$ is the current average. $\bar{x}_{k-1}$ is the previous average. x_k is the current measurement. α is a factor that indicates the weight for the old value, $0 \leq \alpha < 1$.

B. Flow Level Classification

This approach employs the traffic flows between the node and the client to classify the user activities in the Tor traffic. We used two flow exporting tools to generate the flows and to extract the attributes of the flows between the client and the Tor node:

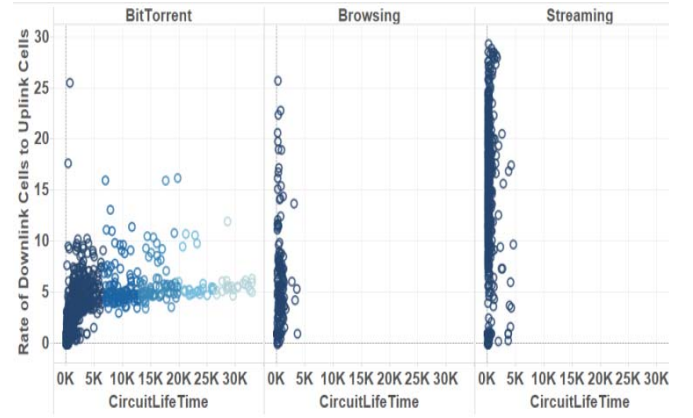


Fig. 6. The Rate of Downlink Cells to Uplink Cells During the Circuit Life Time.

TABLE I. FLOW EXPORTERS ATTRIBUTES

Tool	Number of Attributes	Number of Selected Attributes
Tranalyzer2	91	62
Tcptrace	88	50

1) *Tranalyzer2*: This is an open source tool that generates flows from a captured traffic file or directly by working on the network interface. It is based on the libpcap library. Its output can be exported directly to a text file without the need for an external collector [8]. The output depends on the type of plugins enabled when using Tranalyzer2 to analyze the captured traffic. Choosing the proper plugins is important to generate the features that are useful for the classification task. More information about Tranalyzer2 and other flow exporters can be found in [9].

2) *Tcptrace*: This is a tool used to do a TCP analysis. It can handle different types of captured traffic formats. Different connection graphs can be drawn using Tcptrace. The output can be exported to a text file and a csv file [10]. The attributes vary based on the tools used. Each one of them generates different number of attributes. The number of attributes depends on the options (plugin) enabled in the tool itself. Selecting which attributes to include in the classification of Tor is very important.

It should be stated here that we captured the traffic on our own Tor node and labelled only our own traffic since that is the only ground truth known to us. We differentiate our own traffic using the IP address.

The flow between the client and the node contains many other information that are used by the flow exporters to generate the attributes. For example, the IP address, the port number, the MAC address, time when the flow starts or ends, the sequence number and any information related to the physical client but not the data between the client and the node. Table I shows the number of attributes that Tranalyzer2 and Tcptrace provide in the first column. Second column shows the number of attributes selected by the classifiers from the full set

provided. Tor uses TCP protocol to establish the connection between the user and the relay. So in this work, the attributes that are based on any protocol other than the TCP protocol are excluded. Furthermore, the source port number, the destination port number, the source IP address and the destination IP address attributes are also excluded in the set we used. The IP address is used to label the user activity, since we know which IP address runs which application on our testbed. Last but not the least, we do not employ any payload level information in our attribute set.

V. EXPERIMENTS

A. Setup

In our experiments, we set up a testbed with three clients. These three clients are used to automatically generate the three traffic types: Browsing, Streaming and BitTorrent. All the clients are configured to use our node as the only entry node.

We used iMacro [11] to automate the Browsing and the Streaming activities. Both of them start by searching for random words generated using the English alphabets. They pick randomly one of the sites returned for the search made. In case of Browsing, the iMacro script picks one of the search results, it navigates through the site by clicking a random hyperlink existing on the web site. It keeps moving from one link to another. For each link, the client will stay for a random period of time on that page then move again to another link.

The setting is similar in the streaming activities. The script will move from one video to another. The time spent for watching a video or moving from a link to another is set randomly. The difference between the time set in the browsing and the streaming is that the streaming random time is little longer than the browsing time setting. Deluge [12] is used as the BitTorrent software. We used a proxy to be an interface between the clients' traffic and the Tor node. The socket port in Tor is enabled to listen for the proxy requests.

Tor source code is edited to log only incoming circuit creation requests from our clients. The IP address of each client is used to enable the logging and also to label the circuit as one of the three traffic types. This keeps the privacy of the users who use our node on the Tor network.

B. Data Collection

1) Circuit Level Classification Data

The circuit level classification is based on the cells statistics per circuit to distinguish between each traffic type. Tor source code is edited to relate each circuit to its source IP. Even for the relay cells where Tor will switch the circuit ID, we make Tor logging this ID in its previous circuit ID not the new switched ID. This way we keep track of the uplink and the downlink cells related to each class. The log for each circuit contains the circuit ID, the class, the time when the circuit is created, and the time when the circuit is destroyed. Cells' log contains the class and the circuit ID each cell belongs to, the cell arrival time, and the direction of the cell. The attributes we need in the circuit level classification are then extracted from

the circuits log and the cells log. To maintain the privacy of other users, circuits related to the other users using our node have not been logged.

2) Flow Level Classification Data

PCAP network traces are collected in the entry node machine for the duration of the experiment (10 hours). The required connection information are the traffic between our entry node and each client. To keep the privacy of other users who use our node, a filter is applied to the capturing so only connections from our clients are logged. For the duration of the experiment, we captured 4.2 GB of data.

C. Evaluation Metrics

To obtain a clear picture about the results, we use the same metrics, namely accuracy and F-measure, as used by [1]. This also enables a comparison between the two approaches:

$$\text{Accuracy} = \frac{\text{True Positive (TP)} + \text{True Negative (TN)}}{\text{Number of Instances (N)}}$$

$$\text{Precision} = \frac{\text{True Positive (TP)}}{\text{True Positive (TP)} + \text{False Positive (FP)}}$$

$$\text{Recall} = \frac{\text{True Positive (TP)}}{\text{True Positive (TP)} + \text{False Negative (FN)}}$$

$$\text{F-Measure} = \frac{2 * \text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}}$$

D. Classification Algorithms

We used four supervised ML algorithms: Naïve Bayes, Bayes Net, C4.5 and Random Forest. Naïve Bayes is a simple algorithm based on Bayes theorem. Naïve Bayes [16] does not take into consideration relations between variables while Bayes Net [17] is more general form that makes connections between the related variables. C4.5 [18] is a pruned decision tree based on the information gain of the attributes. Random Forest [19] depends on building multiple decision trees to enhance the decision making. A more detailed information about each one of these well-known classifiers can be found in [20]. We have used Weka [21] to implement each of the classifiers with its default parameters.

TABLE II. CIRCUIT LEVEL CLASSIFICATION RESULTS

Classifier		Accuracy	F-Measure		
			Browsing	Streaming	BitTorrent
Bayes net	SPLIT	98.6%	0.99	0.97	1
	CV	87.7%	0.89	0.82	0.98
Naïve Bayes	SPLIT	98.6%	0.99	0.97	1
	CV	94.5%	0.95	0.88	1
C4.5	SPLIT	100%	1	1	1
	CV	93.2%	0.94	0.83	1
RF	SPLIT	97.2%	0.98	0.97	0.96
	CV	94.9%	0.96	0.88	1

VI. RESULTS AND DISCUSSION

A. Circuit Level Classification Results

We used the circuit level classification with the use of the cell attributes in this approach. The data set consists of 60% browsing, 20% streaming and 20% BitTorrent activities. Table II shows the results for the circuit level classification. In this case, accuracy reaches to 100% when using C4.5 classifier with 70% of the instances as a training set. When 10-Fold cross-validation is used, the best accuracy was ~95% using the random forest classifier. Figure 7 shows the results for the circuit level classification for all the classifiers using the 70-30 training/testing split and the 10-fold cross validation.

B. Flow Level Classification Results

In this case, data set consists of the uniform distribution of the three classes. The number of flows is approximately the same for each class. Even though the amount of data transferred by the Streaming and BitTorrent circuits are larger than the browsing circuits. Table III shows the results when using the uniform distribution of the three classes. The results are 88% to 100% when using 70% of the instances as the training set. When using the 10-Fold cross-validation, results are 86% to 99%.

It should be noted here that the data set employed in [1] consisted of 60% browsing instances, 20% streaming instances and 20% BitTorrent instances. Those authors generated such a data set to mimic the traffic distributions of Tor users. To make our results comparable to theirs, we downsampled our data to generate a data set with the same distributions. Table IV shows the results when our data set consists of the same distributions as [1]. In this case, our results still show high accuracy in both the split and the 10-Fold cross-validation evaluations. Bayes Net classifier achieved 100% in both the split and the 10-Fold cross-validation evaluations. Figure 8 shows the accuracy for all the three classifiers when using the 10-Fold cross-validation evaluations.

C. Accuracy

Table V shows a comparison between the best results achieved in our work and in [1]. Our results show that we improved the performance by 9%.

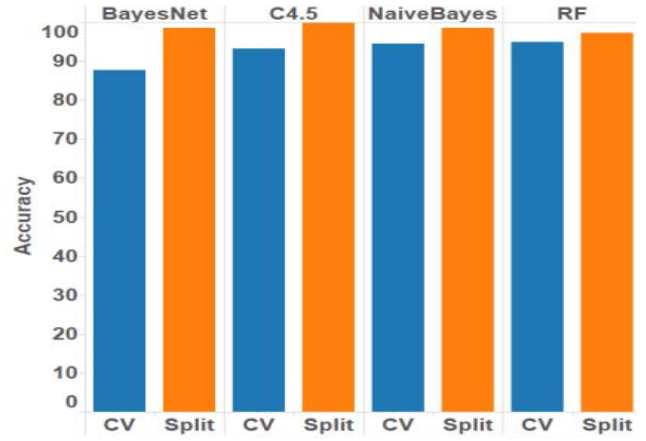


Fig. 7. Circuit Level Classification Accuracy.

TABLE III. FLOW LEVEL CLASSIFICATION RESULTS - UNIFORM CLASSES

	Classifier		Accuracy	F-Measure		
				Browsing	Streaming	BitTorrent
TRANALYZER2	Bayes net	SPLIT	100%	1	1	1
		CV	99.2%	0.99	0.99	1
	Naïve Bayes	SPLIT	94.7%	0.98	0.91	0.94
		CV	93.3%	0.98	0.90	0.93
	C4.5	SPLIT	98.7%	0.98	0.98	1
		CV	97.2%	0.96	0.98	0.98
	RF	SPLIT	98.7%	0.98	0.98	1
		CV	98.8%	0.99	0.98	0.99
TCPTRACE	Bayes net	SPLIT	97.4%	1	0.96	0.96
		CV	97.7%	0.98	0.97	0.99
	Naïve Bayes	SPLIT	97.4%	0.96	1	0.96
		CV	92.2%	0.96	0.89	0.92
	C4.5	SPLIT	94.87%	0.96	0.92	0.97
		CV	96.1%	0.96	0.95	0.98
	RF	SPLIT	97.4%	1	0.96	0.97
		CV	97.7%	0.99	0.96	0.98

In [1], the accuracy was 91% when using functional tree with 10-Fold Cross-Validation. The classification level used was at circuit level. In our work, we achieved 100% by using the circuit level classification with the extended attribute set and also when using the Flow level classification. For the circuit level classification, we achieved this result when using C4.5 with 70% training set. For the Flow level classification, we got 100% accuracy when using Bayes Net, Naïve Bayes and Random Forest classifiers with 70% training data. We also achieved the same result when using Bayes Net classifier with 10-Fold Cross-Validation.

We believe that the achieved improvement in the circuit level classification compared to the result in [1] is due to the features added to the classification. The BitTorrent circuits tend to have longer lifetime than the browsing circuits. There is a small percentage of the BitTorrent circuits that stay active for a short time period and then they are destroyed. These circuits make the BitTorrent's circuit behavior look similar to the browsing's circuit behavior. Therefore, adding the number of cells transferred during the circuit lifetime as an attribute to the attribute set in our work enables the circuits to be more distinguishable. On the other hand, in the browsing circuits the

TABLE IV. FLOW LEVEL CLASSIFICATION RESULTS-DOWNSAMPLED CLASSES

	Classifier		Accuracy	F-Measure		
				Browsing	Streaming	BitTorrent
TRANALYZER2	Bayes net	SPLIT	100%	1	1	1
		CV	100%	1	1	1
	Naïve Bayes	SPLIT	100%	1	1	1
		CV	95.7%	0.99	0.89	0.91
	C4.5	SPLIT	97.6%	0.98	1	0.93
		CV	99.3%	0.99	1	0.98
	RF	SPLIT	100%	1	1	1
		CV	98.6%	1	0.96	0.96
TCPTRACE	Bayes net	SPLIT	81.8%	0.95	0.71	0.73
		CV	93.3%	0.98	0.84	0.90
	Naïve Bayes	SPLIT	77.3%	0.95	0.55	0.71
		CV	82.3%	0.94	0.60	0.73
	C4.5	SPLIT	81.8%	0.82	0.91	0.73
		CV	92.0%	0.94	0.83	0.94
	RF	SPLIT	90.9%	1	0.83	0.83
		CV	94.7%	0.98	0.86	0.93

amount of data per circuit fluctuates between high and low values based on the user's browsing habits and the website content. The same applies to the BitTorrent circuits where not all circuits transfer high amounts of data. Consequently, being able to differentiate between the combined uplink and the downlink data per circuit improves the classification accuracy. Therefore, we added the uplink data alone as an attribute to address this issue. While adding the uplink data is a good indication, there is similarity between the browsing circuits and the streaming circuits in the number of uplink cells. Here, it is useful to employ the rate of downlink to the uplink cells as an attribute to improve the classification performance. In short, we believe that the attribute set that we propose is enabling us to incorporate these important properties of the traffic that we aim to analyze and classify.

Moreover, it should be noted here there is a direct connection between the classification process itself and the user privacy. To the best of our knowledge there is no publicly available Tor data set. This is because for such studies, one needs to capture (collect) the data on the wire. For this reason, we generated and captured the traffic as explained above.

The circuit level classification requires access to the traffic generated by the connections between the user machine and the Tor relay. In the circuit level classification, the access to the relay itself is required. That means classification at the circuit level has its limitations in terms of who can employ it. However, flow level classification does not have these limitations. From the availability prospective, the flow level classification is easier to apply compared to the circuit level classification. The tools used in the flow level classification are available online. These tools include traffic capturing (such as Tcpcdump or Wireshark) and flow exporters to generate the flow statistics (such as Tcptrace or Tranalyzer2).

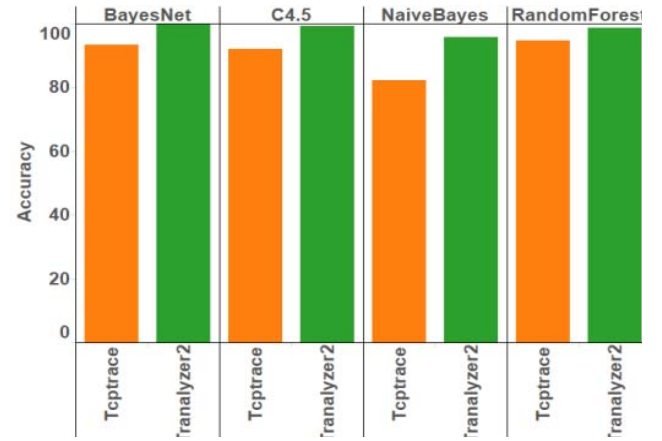


Fig. 8. Flow Level Classification Accuracy using Cross-Validation.

TABLE V. METHODS USED TO ACHIEVE THE BEST ACCURACY

	Accuracy
Results in [1]	91%
	Circuit Level Classification Functional Tree 10-Fold Cross-Validation
Our Results	100%
	Circuit Level Classification C 4.5 Classifier 70% training set
	100%
	Flow Level Classification Bayes Net, Naïve Bayes and Random Forest classifiers 70% training set
	100%
	Flow Level Classification Bayes Net classifiers 10-Fold Cross-Validation

The challenge in the flow level classification is in dealing with the similarities in the traffic at the flow level. Therefore, the selection of the attribute set makes a difference in the performance of the classification. Capturing similar data in circuit level classification (the way we did) that are ready to be classified by the ML algorithms requires additional efforts compared to the flow level classification data. The circuits and cells information in the Tor code maintain a certain level of privacy for the Tor's users. For instance, Cells statistics attribute (CELL_STAT) that are introduced in Tor version 0.2.5.2-alpha are only available when Tor is working in the test mode. This mode works only when TestingTorNetwork value is set in Tor configuration file as described in Tor control protocol [13].

That means using this feature will work only in private networks not the live Tor network. STEM [14] also can provide the user with information about Tor. It provides a library for the user to communicate with Tor using Tor control protocol. It can generate useful information about the circuit and bandwidth usage and so on, but for the same previous example, STEM cannot retrieve the cells statistics by using CELL_STAT event unless Tor is working in the test mode.

In terms of the size of the data used by the two approaches, the flow level classification data is larger than the circuit level classification data. The information in the circuit level is gathered in the form of a text file where each line represents information about a circuit or a cell. Although the number of records is large because of the large number of cells in each circuit, still in a text file this will come to a small size compared to PCAP files. However, for both approaches, when the data are converted to a form that the classifier understands then the size becomes very small.

VII. CONCLUSION AND FUTURE WORK

In this research, we used machine learning algorithms to classify Tor traffic by using two different approaches: circuit level classification and flow level classification. The results show that a high accuracy can be achieved using both approaches. The circuit level classification uses the data captured (collected) at Tor's relay machine. In contrast, the flow level classification uses the data that are captured (collected) anywhere between the user and the Tor's relay machine. Access to the relay machine is not essential in the flow level classification. This enables a very flexible approach to analyze the traffic.

For future work, we aim to use these approaches to classify the user activities online. For both approaches, this will require changes to the attribute set. Therefore, further investigation will be necessary.

ACKNOWLEDGMENT

This research is supported by the Natural Science and Engineering Research Council of Canada (NSERC) grant, and is conducted as part of the Dalhousie NIMS Lab at <http://projects.cs.dal.ca/projectx/>.

REFERENCES

- [1] M. AlSabah, K. Bauer, and I. Goldberg, "Enhancing Tor's performance using real-time traffic classification," in *Proceedings of the 2012 ACM conference on Computer and communications security*, Raleigh, USA, 2012, pp. 73-84.
- [2] J. Barker, P. Hannay, and P. Szewczyk, "Using traffic analysis to identify the second generation onion Router," in *the 9th IFIP International Conference on embedded and ubiquitous computing*, Melbourne, AUS, 2011, pp.72-78.
- [3] A. Panchenko, L. Niessen, A. Zinnen, and T. Engel, "Website fingerprinting in onion routing based anonymization networks," in *Proceedings of the 10th annual ACM workshop on Privacy in the electronic society*, Chicago, USA, 2011, pp.103-104.
- [4] Z. Ling, J. Luo, W. Yu, and X. Fu, "Equal-sized cells mean equal-sized packets in Tor?," in *Proceedings IEEE ICC*, 2011, pp.1 -6
- [5] R. Dingledine, N. Mathewson, and P. Syverson, "Tor: the second-generation onion router," in *Proceedings of the 13th conference on USENIX Security Symposium - Volume 13*. USENIX Association, 2004, pp. 21-21.
- [6] R. Dingledine, and N. Mathewson. *Tor protocol specification* [Online]. Available: https://gitweb.torproject.org/torspec.git?a=blob_plain;hb=HEAD;f=torspec.txt
- [7] *Exponentially weighted moving average* [Online]. Available: <http://lorien.ncl.ac.uk/ming/filter/filewma.htm>
- [8] *TRANALYZER2* [Online]. Available: <http://tranalyzer.com/>
- [9] F. Haddadi, and N. Zincir-Heywood. *Benchmarking the Effect of Flow Exporters and Protocol Filters on Botnet Traffic Classification* [Online]. Available: <https://www.cs.dal.ca/research/techreports/cs-2014-02>
- [10] *TCPTTRACE* [Online]. Available: <http://www.tcptrace.org/>
- [11] *iMacros* [Online]. Available: <http://imacros.net/overview>
- [12] *DELUGE* [Online]. Available: <http://deluge-torrent.org/>
- [13] *Tor control protocol* [Online]. Available: https://gitweb.torproject.org/torspec.git?a=blob_plain;hb=HEAD;f=control-spec.txt
- [14] *STEM* [Online]. Available: <https://stem.torproject.org/>
- [15] *Alexa* [Online]. Available: <http://www.alexa.com/topsites>
- [16] G. John, and P. Langley, "Estimating continuous distributions in bayesian classifiers," in *Proceedings of the Eleventh Conference on Uncertainty in Artificial Intelligence*, San Mateo, USA, 1995, pp. 338-345.
- [17] G. Cooper, and E. Herskovits, "A Bayesian method for constructing Bayesian belief networks from databases," in *Proceedings of the Seventh conference on Uncertainty in Artificial Intelligence*, Los Angeles, USA, 1991, pp.86-94.
- [18] J. Ross Quinlan, C4.5: Program for Machine Learning, Morgan Kaufmann Inc., San Francisco, CA, 1993.
- [19] L. Breiman, Random Forests, Machine Learning, vol.45, no.1, pp.5-32, 2001.
- [20] P. Flach, Machine Learning: The Art and Science of Algorithms that Make Sense of Data, Cambridge University Press, New York, NY, 2012.
- [21] M. Hall, E. Frank, G. Holmes, B. Pfahringer, P. Reutemann, and I. Witten, "The WEKA data mining software: an update," SIGKDD Explorations, vol. 11, no. 1, pp. 10-18, 2009.